University of Chittagong
7th Semester B.Sc. (Engg.) Examination 2019
Department of Computer Science & Engineering

Course Code: CSE 715 Course Title: Machine Learning

Fully Worked Solutions

Answer six questions taking any three from each section. Full Marks: $52\frac{1}{2}$*, Time: 4 Hours.*

Section A

Q0.1 Apriori Algorithm

(a) Anti-monotone property of support – 3 marks

Question: The Apriori algorithm makes use of prior knowledge of subset support properties.
(i) Prove that all nonempty subsets of a frequent itemset must also be frequent.
(ii) Prove that the support of any nonempty subset s' of itemset s must be at least as great as the support of s.

Solution:

(i) Proof: Let X be a frequent itemset, i.e. $\text{support}(X) \geq min_sup$. Let Y be any nonempty subset of X, so $Y \subseteq X$.

By definition, the support of an itemset is the number (or fraction) of transactions that contain *all* items of that itemset. If a transaction T contains every item of X, then, because $Y \subseteq X$, T must also contain every item of Y. Hence:

$$\{T : X \subseteq T\} \subseteq \{T : Y \subseteq T\}$$

Taking cardinalities (or dividing by $|D|$ for the fractional form):

$$\text{support}(Y) \geq \text{support}(X) \geq min_sup$$

Therefore Y is also frequent. Since Y was an arbitrary nonempty subset of X, **every nonempty subset of a frequent itemset is frequent**. ■

(ii) Proof: Let s be an itemset and s' a nonempty subset of s, i.e. $s' \subseteq s$. As above, any transaction that contains s must also contain s' (because every item in s' is also in s). Thus the set of transactions supporting s is a subset of the set of transactions supporting s':

$$\{T : s \subseteq T\} \subseteq \{T : s' \subseteq T\}$$

so

$$\text{count}(s') \geq \text{count}(s) \implies \text{support}(s') \geq \text{support}(s).$$

This is exactly the *anti-monotone (downward closure)* property that Apriori exploits to prune candidates. ■

(b) Apriori vs. FP-Growth efficiency – 1.75 marks

Question: Apriori and FP-Growth are two famous algorithms for frequent itemset generation. Compare the efficiency of the two mining processes.

Solution:

Criterion	Apriori	FP-Growth
Approach	Candidate generate-and-test	Pattern growth (divide and conquer), no candidate generation
Database scans	Multiple scans (one per candidate-set size k)	Only 2 scans total
Candidates generated	Very large number of candidates, especially for long frequent itemsets	None – mines directly from the compressed FP-tree
Memory/data structure	Stores candidate itemsets and their counts	Builds a compact FP-tree (prefix-tree) representation
Performance on low support / dense data	Degrades sharply (combinatorial explosion of candidates)	Scales much better
Overall efficiency	Slower, especially for large/dense datasets	Generally faster, more scalable

Conclusion: FP-Growth is generally more efficient than Apriori because it avoids the costly candidate generation-and-test step and needs only two database scans, whereas Apriori's cost grows quickly with the number of candidates and the number of passes over the database.

(c) Apriori on the transaction database – 4 marks

Question: A database has eight transactions. Let $min_sup = 2$. Find all frequent itemsets using the Apriori Algorithm.

TID	1	2	3	4	5	6	7	8
Items	K,C,A	A,M,D	C,K,D	K,D,A	A,C,S	D,A,S	K,S,A	A,M,K

Solution:

Step 1: Count 1-itemsets ($C_1 \to L_1$), $min_sup = 2$

Itemset	{A}	{C}	{D}	{K}	{M}	{S}
Support count	7	3	4	5	2	3

All six 1-itemsets have support ≥ 2, so $L_1 = \{A, C, D, K, M, S\}$ (all frequent).

Step 2: Generate candidate 2-itemsets (C_2) and count support

Pair	Count	Frequent?	Pair	Count	Frequent?
AC	2	Yes	DK	2	Yes
AD	3	Yes	DM	1	No
AK	4	Yes	DS	1	No
AM	2	Yes	KM	1	No
AS	3	Yes	KS	1	No
CD	1	No	MS	0	No
CK	2	Yes	CM	0	No
CS	1	No			

$$L_2 = \{AC(2),\ AD(3),\ AK(4),\ AM(2),\ AS(3),\ CK(2),\ DK(2)\}$$

Step 3: Generate candidate 3-itemsets (C_3) by joining L_2, then prune
Joining pairs from L_2 that share the same first item gives the candidates: $ACD, ACK, ACM, ACS, ADK, ADM,$
Pruning (a candidate is kept only if *all* of its 2-item subsets are in L_2):

Candidate	2-subsets check	Result
ACD	AC✓, AD✓, CD**(not in L_2)**	pruned
ACK	AC✓, AK✓, CK✓	**kept**
ACM	AC✓, AM✓, CM(not in L_2)	pruned
ACS	AC✓, AS✓, CS(not in L_2)	pruned
ADK	AD✓, AK✓, DK✓	**kept**
ADM	AD✓, AM✓, DM(not in L_2)	pruned
ADS	AD✓, AS✓, DS(not in L_2)	pruned
AKM	AK✓, AM✓, KM(not in L_2)	pruned
AKS	AK✓, AS✓, KS(not in L_2)	pruned
AMS	AM✓, AS✓, MS(not in L_2)	pruned

So $C_3 = \{ACK,\ ADK\}$.

Counting support: ACK appears only in TID 1 (count = 1); ADK appears only in TID 4 (count = 1). Both fall below $min_sup = 2$, so $L_3 = \varnothing$ and the algorithm terminates.

Final Answer – all frequent itemsets:

$L_1 \cup L_2 = \{A(7),\ C(3),\ D(4),\ K(5),\ M(2),\ S(3),\ AC(2),\ AD(3),\ AK(4),\ AM(2),\ AS(3),\ CK(2),\ DK(2)\}$

That is **13 frequent itemsets** in total (6 frequent 1-itemsets + 7 frequent 2-itemsets), and no frequent 3-itemset exists.

Q0.2 Decision Trees

(a) Why Random Forest may improve accuracy – 2.75 marks

Question: State why Random Forest may improve the accuracy of decision tree induction.

Solution:

A single decision tree is grown deterministically on the whole training set and tends to have low bias but *high variance*: it can overfit and become very sensitive to small changes in the data. Random Forest is an *ensemble* of many decision trees where:

- Each tree is trained on a different **bootstrap sample** (bagging) of the training data.
- At every split, only a **random subset of features** is considered, which decorrelates the trees (different trees pick up different patterns and make different mistakes).
- The final prediction is obtained by **majority voting** (classification) or averaging (regression) over all trees.

Because the individual trees' errors are largely uncorrelated, averaging/voting cancels out much of the variance while keeping bias low, so the ensemble generalizes better and achieves higher accuracy than any single decision tree, especially on noisy or high-dimensional data.

(b) Information Gain and Gini Index for Table I – 6 marks

Question: Consider Table I for a binary class problem (10 records, attributes A, B, Class Label). (i) Calculate the information gain when splitting on A and B; which attribute would the decision tree choose? (ii) Calculate the gain in the Gini index when splitting on A and B; which attribute would the decision tree choose?

A	B	Class
T	F	+
T	T	+
T	T	+
T	F	–
T	T	+
F	F	–
F	F	–
F	F	–
T	T	–
T	F	–

Solution:

Total records = 10: **4 (+)** and **6 (−)**, so $p_+ = 0.4,\ p_- = 0.6$.

Parent entropy:

$$Entropy(parent) = -0.4\log_2 0.4 - 0.6\log_2 0.6 = 0.5288 + 0.4422 = \mathbf{0.971}$$

Split on A:

	#(+)	#(−)	Total
$A = T$	4	3	7
$A = F$	0	3	3

$$Entropy(A{=}T) = -\tfrac{4}{7}\log_2\tfrac{4}{7} - \tfrac{3}{7}\log_2\tfrac{3}{7} = 0.4614 + 0.5240 = 0.9852$$

$$Entropy(A{=}F) = 0 \quad \text{(pure node, all } -\text{)}$$

$$Entropy_{split}(A) = \tfrac{7}{10}(0.9852) + \tfrac{3}{10}(0) = 0.6897$$

$$\boxed{Gain(A) = 0.971 - 0.6897 = \mathbf{0.281}}$$

Split on B:

	#(+)	#(−)	Total
$B = T$	3	1	4
$B = F$	1	5	6

$$Entropy(B{=}T) = -\tfrac{3}{4}\log_2\tfrac{3}{4} - \tfrac{1}{4}\log_2\tfrac{1}{4} = 0.3113 + 0.5 = 0.8113$$

$$Entropy(B{=}F) = -\tfrac{1}{6}\log_2\tfrac{1}{6} - \tfrac{5}{6}\log_2\tfrac{5}{6} = 0.4309 + 0.2192 = 0.6501$$

$$Entropy_{split}(B) = \tfrac{4}{10}(0.8113) + \tfrac{6}{10}(0.6501) = 0.3245 + 0.3901 = 0.7146$$

$$\boxed{Gain(B) = 0.971 - 0.7146 = \mathbf{0.256}}$$

Decision (Information Gain): Since $Gain(A) = 0.281 > Gain(B) = 0.256$, the tree induction algorithm would **split on A** (higher information gain ⇒ greater reduction in entropy).

Gini Index

$$Gini(parent) = 1 - (p_+^2 + p_-^2) = 1 - (0.4^2 + 0.6^2) = 1 - 0.52 = 0.48$$

Split on A:

$$Gini(A{=}T) = 1 - \left[(\tfrac{4}{7})^2 + (\tfrac{3}{7})^2\right] = 1 - \tfrac{25}{49} = 0.4898, \qquad Gini(A{=}F) = 1 - (0^2 + 1^2) = 0$$

$$Gini_{split}(A) = \tfrac{7}{10}(0.4898) + \tfrac{3}{10}(0) = 0.3429$$

$$\boxed{Gain_{Gini}(A) = 0.48 - 0.3429 = \mathbf{0.137}}$$

Split on B:

$$Gini(B{=}T) = 1 - \left[(\tfrac{3}{4})^2 + (\tfrac{1}{4})^2\right] = 1 - \tfrac{10}{16} = 0.375, \qquad Gini(B{=}F) = 1 - \left[(\tfrac{1}{6})^2 + (\tfrac{5}{6})^2\right] = 1 - \tfrac{26}{36} = 0.2778$$

$$Gini_{split}(B) = \tfrac{4}{10}(0.375) + \tfrac{6}{10}(0.2778) = 0.15 + 0.1667 = 0.3167$$

$$\boxed{Gain_{Gini}(B) = 0.48 - 0.3167 = \mathbf{0.163}}$$

Decision (Gini Index): Since the weighted Gini of B (0.317) is lower than that of A (0.343), i.e. $Gain_{Gini}(B) = 0.163 > Gain_{Gini}(A) = 0.137$, the tree induction algorithm would **split on B** when Gini index is used.

Note: Interestingly, the two impurity measures disagree here – Information Gain prefers A while the Gini Index prefers B. This is a good illustration that the choice of splitting criterion can affect which attribute a decision-tree algorithm selects.

Q0.3 Naive Bayes Classifier

(a) Bayesian classifiers are statistical classifiers – 1.75 marks

Question: Bayesian classifiers are statistical classifiers. Justify your answer.

Solution:

Bayesian classifiers predict, for a given tuple X, the *probability* that X belongs to each class C_i, and assign X to the class with the highest posterior probability. This is done using **Bayes' theorem**:

$$P(C_i \mid X) = \frac{P(X \mid C_i)\, P(C_i)}{P(X)}$$

Every term here – the prior $P(C_i)$, the class-conditional likelihood $P(X \mid C_i)$, and the posterior $P(C_i \mid X)$ – is estimated purely from the **frequency counts / probability distributions** observed in the training data, not from explicit logical rules or geometric boundaries. Classification is made by a statistical decision rule (Maximum A Posteriori, MAP): choose C_i that maximizes $P(C_i \mid X)$. Because the entire method is grounded in probability theory and statistical estimation from data, Bayesian classifiers are correctly called **statistical classifiers**.

(b) Class-conditional independence – 2 marks

Question: Why is naive Bayesian classification known as class-conditional independence?

Solution:

Naive Bayes makes the simplifying ("naive") assumption that, *given the class label*, the attributes are conditionally independent of one another – i.e. knowing the class already explains all the correlation among attributes, so no attribute gives extra information about another once the class is known. Formally:

$$P(X \mid C_i) = P(x_1, x_2, \ldots, x_n \mid C_i) = \prod_{k=1}^{n} P(x_k \mid C_i)$$

This assumption is called **class-conditional independence** because independence is assumed *conditioned on the class*, not unconditionally – the attributes may well be correlated overall, but are treated as independent *within* each class. This assumption drastically simplifies estimating $P(X \mid C_i)$ (only need $P(x_k \mid C_i)$ for each attribute) and is the defining ("naive") feature of the classifier.

(c) Naive Bayes – Play Tennis Prediction – 5 marks

Question: Table II gives 14 records of Outlook, Temperature, Humidity, Wind, Play. Today is Sunny, Mild, High and Strong. Using Naive Bayesian Classifier, determine if anyone will play tennis today.

Solution:

From the 14 records: $Play = Yes$: 9 instances, $Play = No$: 5 instances.

$$P(Yes) = \frac{9}{14} = 0.643, \qquad P(No) = \frac{5}{14} = 0.357$$

Conditional probabilities needed (query: Outlook=Sunny, Temp=Mild, Humidity=High, Wind=Strong):

	Given Play=Yes (9 total)	Given Play=No (5 total)
P(Sunny)	$2/9 = 0.222$	$3/5 = 0.600$
P(Mild)	$4/9 = 0.444$	$2/5 = 0.400$
P(High)	$3/9 = 0.333$	$4/5 = 0.800$
P(Strong)	$3/9 = 0.333$	$3/5 = 0.600$

Posterior (unnormalized) for Yes:

$$P(Yes) \cdot P(\text{Sunny}|Yes) \cdot P(\text{Mild}|Yes) \cdot P(\text{High}|Yes) \cdot P(\text{Strong}|Yes)$$

$$= 0.643 \times 0.222 \times 0.444 \times 0.333 \times 0.333 = \mathbf{0.00706}$$

Posterior (unnormalized) for No:

$$P(No) \cdot P(\text{Sunny}|No) \cdot P(\text{Mild}|No) \cdot P(\text{High}|No) \cdot P(\text{Strong}|No)$$

$$= 0.357 \times 0.600 \times 0.400 \times 0.800 \times 0.600 = \mathbf{0.04114}$$

Normalizing (dividing each by their sum $0.00706 + 0.04114 = 0.04820$):

$$P(Yes \mid X) = \frac{0.00706}{0.04820} = 14.6\%, \qquad P(No \mid X) = \frac{0.04114}{0.04820} = 85.4\%$$

Since $P(No \mid X) \gg P(Yes \mid X)$, the Naive Bayes classifier predicts:

$$\boxed{\textbf{Play = No – nobody will play tennis today.}}$$

Q0.4 Classifier Evaluation

(a) Accuracy with multi-label objects – 2.75 marks

Question: It is difficult to assess classification accuracy when individual data objects may belong to more than one class at a time. Comment on what criteria you would use to compare different classifiers modeled after the same data.

Solution:

When an object can genuinely belong to several classes simultaneously (multi-label classification), simple "accuracy" (exact match with one true label) becomes too strict and unreliable, since a prediction that is *partially* correct is counted as completely wrong. More appropriate criteria include:

- **Hamming Loss** – fraction of individual labels that are predicted incorrectly (a per-label, partial-credit measure).
- **Precision, Recall, and F1-measure** computed per label and then **macro-averaged** (average across labels, treats all labels equally) or **micro-averaged** (aggregate all label decisions, favors frequent labels).
- **Jaccard Similarity / Index** between the predicted label set and the actual label set of each object – rewards partial overlap.
- **Subset (Exact-Match) Accuracy** – for cases where an exact match of the entire label set is still required.
- **Label ranking measures** (e.g., one-error, coverage, average precision) when the classifier outputs a ranked list/score per label.

These criteria give *partial credit* for overlapping/partially-correct predictions instead of requiring a single exact class match, making them far more suitable for comparing classifiers on data where objects can belong to multiple classes.

(b) Interpolated Recall–Precision Curve – 3 marks

Question: Select between two prediction models M1 and M2. Smooth both models' Recall-Precision (R-P) using interpolation between 0–1, and draw the R-P curve for model selection.

M1					
Recall	0.20	0.40	0.70	0.90	1.0
Precision	0.12	0.40	0.67	0.75	0.88

M2					
Recall	0.10	0.30	0.86	0.96	1.0
Precision	0.19	0.45	0.76	0.85	0.98

Solution:

The standard **interpolation rule** for a Recall-Precision curve defines, at any recall level r:

$$p_{interp}(r) = \max_{r' \geq r} p(r')$$

i.e. the interpolated precision at recall r is the *highest* precision observed at any recall $\geq r$. This is applied at the 11 standard recall levels $r = 0.0, 0.1, 0.2, \ldots, 1.0$.

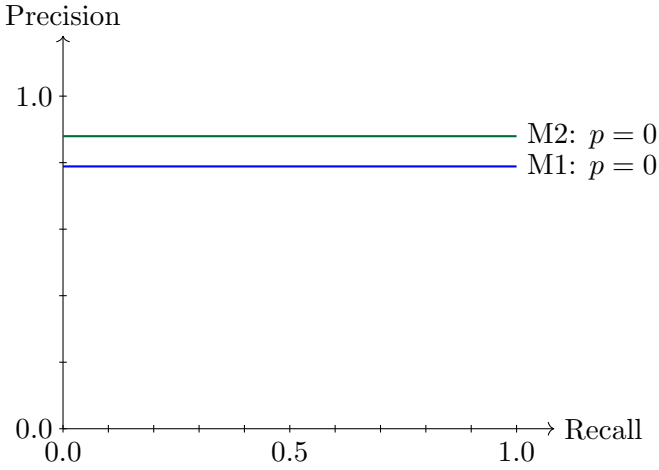
M1: the raw points $(0.20, 0.12), (0.40, 0.40), (0.70, 0.67), (0.90, 0.75), (1.0, 0.88)$ have precision *strictly increasing* with recall. So for every recall level $r \in [0, 1.0]$, the maximum precision at recall $\geq r$ is always the value at the highest recall point, 0.88:

$$p^{M1}_{interp}(r) = 0.88 \quad \text{for all } r = 0.0, 0.1, \ldots, 1.0$$

M2: likewise $(0.10, 0.19), (0.30, 0.45), (0.86, 0.76), (0.96, 0.85), (1.0, 0.98)$ is also strictly increasing, so:

$$p^{M2}_{interp}(r) = 0.98 \quad \text{for all } r = 0.0, 0.1, \ldots, 1.0$$

Interpolated R-P curve (both are horizontal lines):



Model selection: Because $p_{interp}^{M2}(r) = 0.98 > p_{interp}^{M1}(r) = 0.88$ at *every* recall level, the interpolated R-P curve of **M2 dominates M1 across the entire recall range** (M2's curve lies completely above M1's). Hence **M2 is the preferred model** – it achieves higher precision at every level of recall.

(c) Cross Validation vs. Stratified Cross Validation ($k = 2$) – 3 marks

Question: Construction and evaluation of a classifier require partitioning labeled data into a training set and a test set. Sketch (i) Cross Validation ($k = 2$) and (ii) Stratified Cross Validation ($k = 2$) for the given data set (10 records, tid 1–10, Class Label E1/E2, where E1: tid 1,2,3,5,6,9 (6 records) and E2: tid 4,7,8,10 (4 records)).

Solution:

The full data set D has 10 records with class distribution $E1 : E2 = 6 : 4 = 3 : 2$.

(i) 2-fold Cross Validation: D is split into two folds of equal size *without* regard to class proportions – e.g. a simple sequential split:

$$D_1 = \{1, 2, 3, 4, 5\}, \qquad D_2 = \{6, 7, 8, 9, 10\}$$

$D_1 = \{1, 2, 3, 4, 5\}$	$D_2 = \{6, 7, 8, 9, 10\}$
Round 1: Train on D_2, Test on D_1	**Round 2:** Train on D_1, Test on D_2

Overall estimated accuracy = average of the accuracies obtained in Round 1 and Round 2. Note D_1 has 4 E1 + 1 E2, and D_2 has 2 E1 + 3 E2 – the class ratio is **not** preserved in each fold.

(ii) Stratified 2-fold Cross Validation: folds are built so that each fold preserves the original class ratio 3:2 (since fold size = 5, each fold should get 3 of class E1 and 2 of class E2):

E1 records $\{1, 2, 3, 5, 6, 9\} \rightarrow \{1, 2, 3\}$ and $\{5, 6, 9\}$, E2 records $\{4, 7, 8, 10\} \rightarrow \{4, 7\}$ and $\{8, 10\}$

$$D_1^{strat} = \{1, 2, 3, 4, 7\}\ (3\,E1 + 2\,E2), \qquad D_2^{strat} = \{5, 6, 9, 8, 10\}\ (3\,E1 + 2\,E2)$$

$D_1^{strat} = \{1, 2, 3, 4, 7\}$ (3 E1, 2 E2)

$D_2^{strat} = \{5, 6, 9, 8, 10\}$ (3 E1, 2 E2)

Round 1:
Train on D_2^{strat}, Test on D_1^{strat}

Round 2:
Train on D_1^{strat}, Test on D_2^{strat}

Both folds here have exactly the same $E1$:$E2$ ratio (3:2) as the whole data set D. This is the key difference: **stratified CV explicitly maintains class proportions in every fold**, giving a less biased, more representative accuracy estimate – especially important when classes are imbalanced.

Section B

Q0.5 K-Means Clustering (CGPA)

Question: Cluster the students' CGPA {A=5.45, B=3.88, C=2.78, D=1.25, E=2.2, F=3.80} into 3 clusters using Manhattan distance. Initial centers: B, D, F. Use k-means to show only the final three clusters.

Solution:

Since the data is one-dimensional, Manhattan distance is simply $|x - c|$. Initial centroids: $c_1 = B = 3.88$, $c_2 = D = 1.25$, $c_3 = F = 3.80$.

Iteration 1 (assign to nearest of c_1=3.88, c_2=1.25, c_3=3.80):

Point	$\lvert x - c_1 \rvert$	$\lvert x - c_2 \rvert$	$\lvert x - c_3 \rvert$	Assigned
A=5.45	1.57	4.20	1.65	c_1
B=3.88	0.00	2.63	0.08	c_1
C=2.78	1.10	1.53	1.02	c_3
D=1.25	2.63	0.00	2.55	c_2
E=2.2	1.68	0.95	1.60	c_2
F=3.80	0.08	2.55	0.00	c_3

Clusters: $\{A, B\}$, $\{D, E\}$, $\{C, F\}$. New centroids (mean): $c_1 = \frac{5.45+3.88}{2} = 4.665$, $c_2 = \frac{1.25+2.2}{2} = 1.725$, $c_3 = \frac{2.78+3.80}{2} = 3.29$.

Iteration 2 (reassign with c_1=4.665, c_2=1.725, c_3=3.29):

Point	$\lvert x - c_1 \rvert$	$\lvert x - c_2 \rvert$	$\lvert x - c_3 \rvert$	Assigned
A=5.45	0.785	3.725	2.16	c_1
B=3.88	0.785	2.155	0.59	c_3
C=2.78	1.885	1.055	0.51	c_3
D=1.25	3.415	0.475	2.04	c_2
E=2.2	2.465	0.475	1.09	c_2
F=3.80	0.865	2.075	0.51	c_3

Clusters: $\{A\}$, $\{D, E\}$, $\{B, C, F\}$ (B moved). New centroids: $c_1 = 5.45$, $c_2 = 1.725$, $c_3 = \frac{3.88+2.78+3.80}{3} = 3.4867$.

Iteration 3 (reassign with c_1=5.45, c_2=1.725, c_3=3.4867):

Point	$\lvert x - c_1 \rvert$	$\lvert x - c_2 \rvert$	$\lvert x - c_3 \rvert$	Assigned
A=5.45	0.00	3.725	1.963	c_1
B=3.88	1.57	2.155	0.393	c_3
C=2.78	2.67	1.055	0.707	c_3
D=1.25	4.20	0.475	2.237	c_2
E=2.2	3.25	0.475	1.287	c_2
F=3.80	1.65	2.075	0.313	c_3

Clusters $\{A\}$, $\{D, E\}$, $\{B, C, F\}$ are **unchanged** from Iteration 2 $\Rightarrow$ the algorithm has **converged**.

Final Clusters: $\{A = 5.45\}$ $\{D = 1.25,\ E = 2.2\}$ $\{B = 3.88,\ C = 2.78,\ F = 3.80\}$

Q5(b) AGNES – Dendrogram (Single-Linkage)

Question: Use the dissimilarity matrix to perform AGNES. Show the results by drawing a dendrogram indicating the order in which the points are merged.

	P1	P2	P3	P4	P5
P1	0.00	0.10	0.41	0.55	0.35
P2	0.10	0.00	0.64	0.47	0.98
P3	0.41	0.64	0.00	0.44	0.85
P4	0.55	0.47	0.44	0.00	0.76
P5	0.35	0.98	0.85	0.76	0.00

Solution:

AGNES (Agglomerative Nesting) starts with every point as its own cluster and repeatedly merges the two closest clusters. We use **single-linkage** (minimum distance between clusters), the standard default.

Step 1: Smallest entry in the matrix is $d(P1, P2) = 0.10 \Rightarrow$ merge $\{P1, P2\}$.

Step 2: Update distances using $d(\{P1, P2\}, X) = \min(d(P1, X), d(P2, X))$:

$d(\{P1, P2\}, P3) = \min(0.41, 0.64) = 0.41, \quad d(\{P1, P2\}, P4) = \min(0.55, 0.47) = 0.47, \quad d(\{P1, P2\}, P5) = \min(0.3$

Smallest is $d(\{P1, P2\}, P5) = 0.35 \Rightarrow$ merge $\{P1, P2, P5\}$.

Step 3: Update:

$$d(\{P1, P2, P5\}, P3) = \min(0.41, 0.85) = 0.41, \quad d(\{P1, P2, P5\}, P4) = \min(0.47, 0.76) = 0.47$$

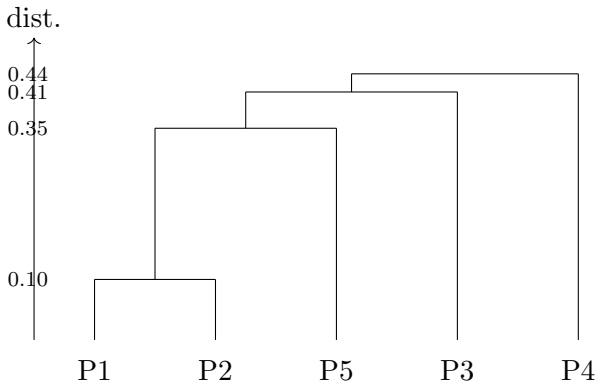
Smallest is $d(\{P1, P2, P5\}, P3) = 0.41 \Rightarrow$ merge $\{P1, P2, P3, P5\}$.

Step 4: Only P4 remains: $d(\{P1, P2, P3, P5\}, P4) = \min(0.47, 0.44) = 0.44 \Rightarrow$ merge everything into $\{P1, P2, P3, P4, P5\}$.

Merge order summary:

Step	Merge	Height (distance)
1	$P1 + P2$	0.10
2	$(P1, P2) + P5$	0.35
3	$(P1, P2, P5) + P3$	0.41
4	$(P1, P2, P3, P5) + P4$	0.44

Dendrogram:



Q0.6 Backpropagation Neural Network

(a) Improving interpretability – 2 marks

Question: How can you improve the interpretability of the learned network in backpropagation?

Solution:

- **Network pruning:** remove weak links (small-magnitude weights) and redundant hidden units that contribute little to the output, leaving a smaller, easier-to-read network that highlights the important input-output relationships.
- **Rule extraction:** analyze the trained weights/activation patterns and convert them into human-readable if-then classification rules.
- **Sensitivity analysis:** vary one input at a time to see how much the output changes, revealing which input attributes matter most.

(b) Connectionist learning – 2 marks

Question: Why is neural network learning referred to as connectionist learning?

Solution:

A neural network consists of many simple processing units (neurons) joined by weighted **connections**, similar in spirit to biological neurons connected by synapses. Knowledge is not stored as explicit symbolic rules but is **distributed across the connection weights** themselves, and "learning" consists purely of adjusting these weights (e.g., via backpropagation) so that the network produces the desired input-output mapping. Because the representation and the learning process are both centered on the network of *connections* rather than symbolic logic, this paradigm is called **connectionist learning**.

(c) Why use a squashing function – 1.75 marks

Question: Why do we use a squashing function in backpropagation?

Solution:

- **Non-linearity:** without a non-linear activation, stacking multiple layers would collapse to an equivalent single linear function, so the network could not model complex, non-linear decision boundaries.
- **Bounded output:** functions like the sigmoid "squash" the weighted sum into a fixed range (e.g., $(0, 1)$), keeping activations from growing unbounded as they pass through many layers.
- **Differentiability:** backpropagation needs the derivative of the activation function to apply the chain rule when computing gradients; squashing functions such as the sigmoid are smooth and easily differentiable.

(d) Design MLP and one backpropagation iteration – 3 marks

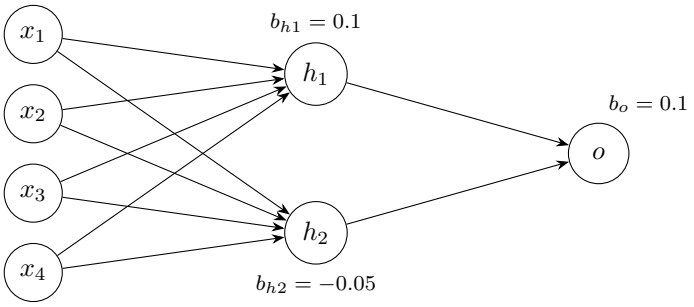
Question: For the given table, design a multilayer feed-forward neural network and show weight values after one iteration of backpropagation, given the training instance X = (Body Temperature=warm, Gives Birth=yes, Four-legged=no, Hibernates=yes). Indicate initial weight values, biases, and learning rate used.

Solution:

Encoding: x_1 = Body Temp (warm= 1/cold= 0), x_2 = Gives Birth (yes= 1/no= 0), x_3 = Four-legged (yes= 1/no= 0), x_4 = Hibernates (yes= 1/no= 0). For X: $x_1 = 1,\ x_2 = 1,\ x_3 = 0,\ x_4 = 1$.

This attribute pattern (warm-blooded, gives birth, not four-legged, hibernates) matches a mammal such as a bat, so the target/desired output is $d = 1$ (Class = yes/mammal).

Architecture chosen: 4 input neurons → 2 hidden neurons (h_1, h_2, sigmoid) → 1 output neuron (o, sigmoid), fully connected with bias at the hidden and output layers, learning rate $\eta = 0.5$.



Initial weights (chosen, as none were given):

Input→ h_1:	w_{11}=0.20, w_{21}=0.30, w_{31}=−0.10, w_{41}=0.15,	b_{h1}=0.10
Input→ h_2:	w_{12}=0.25, w_{22}=−0.20, w_{32}=0.10, w_{42}=0.30,	b_{h2}=−0.05
Hidden→output:	v_1=0.40, v_2=0.20, b_o=0.10	

Learning rate $\eta = 0.5$. Activation: sigmoid $f(z) = \dfrac{1}{1+e^{-z}}$.

Forward Pass:

$$net_{h1} = 1(0.20) + 1(0.30) + 0(-0.10) + 1(0.15) + 0.10 = 0.75 \;\Rightarrow\; h_1 = f(0.75) = 0.6792$$

$$net_{h2} = 1(0.25) + 1(-0.20) + 0(0.10) + 1(0.30) - 0.05 = 0.30 \;\Rightarrow\; h_2 = f(0.30) = 0.5744$$

$$net_o = 0.6792(0.4) + 0.5744(0.2) + 0.10 = 0.4866 \;\Rightarrow\; o = f(0.4866) = 0.6193$$

Error $= d - o = 1 - 0.6193 = 0.3807$.

Backward Pass (deltas):

$$\delta_o = o(1-o)(d-o) = 0.6193(0.3807)(0.3807) = 0.0898$$

$$\delta_{h1} = h_1(1-h_1)(\delta_o v_1) = 0.6792(0.3208)(0.0898 \times 0.4) = 0.00783$$

$$\delta_{h2} = h_2(1-h_2)(\delta_o v_2) = 0.5744(0.4256)(0.0898 \times 0.2) = 0.00439$$

Weight updates ($\Delta w = \eta\, \delta\, \text{input}$):

Weight	Update	New value
v_1 (hidden→output)	$\Delta v_1 = 0.5(0.0898)(0.6792) = 0.0305$	$v_1 = 0.4 + 0.0305 = 0.4305$
v_2	$\Delta v_2 = 0.5(0.0898)(0.5744) = 0.0258$	$v_2 = 0.2 + 0.0258 = 0.2258$
b_o	$\Delta b_o = 0.5(0.0898) = 0.0449$	$b_o = 0.10 + 0.0449 = 0.1449$
w_{11}	$\Delta = 0.5(0.00783)(1) = 0.00391$	$w_{11} = 0.2039$
w_{21}	$\Delta = 0.5(0.00783)(1) = 0.00391$	$w_{21} = 0.3039$
w_{31}	$\Delta = 0.5(0.00783)(0) = 0$	$w_{31} = -0.10$ (unchanged, x_3=0)
w_{41}	$\Delta = 0.5(0.00783)(1) = 0.00391$	$w_{41} = 0.1539$
b_{h1}	$\Delta = 0.5(0.00783) = 0.00391$	$b_{h1} = 0.1039$
w_{12}	$\Delta = 0.5(0.00439)(1) = 0.00220$	$w_{12} = 0.2522$
w_{22}	$\Delta = 0.5(0.00439)(1) = 0.00220$	$w_{22} = -0.1978$
w_{32}	$\Delta = 0.5(0.00439)(0) = 0$	$w_{32} = 0.10$ (unchanged, x_3=0)
w_{42}	$\Delta = 0.5(0.00439)(1) = 0.00220$	$w_{42} = 0.3022$
b_{h2}	$\Delta = 0.5(0.00439) = 0.00220$	$b_{h2} = -0.0478$

These are the updated weights and biases after one full iteration (one forward pass + one backward pass) of backpropagation for the training instance X.

Q0.7 Limitations of K-Means & Clustering Applications

(a) Limitations of k-means – 3 marks

Question: Illustrate the limitation of the k-means algorithm.

Solution:

- Requires the number of clusters K to be **specified in advance**, which is often unknown.
- **Sensitive to the initial choice of centroids** – different initializations can converge to different final clusterings (different local optima).
- Works well only for clusters that are roughly **spherical/convex, similar in size and density**; it performs poorly on clusters with non-convex, irregular shapes, or very different sizes/densities.
- **Sensitive to outliers and noise** because the mean can be heavily skewed by extreme values.
- Defined only where a "mean" makes sense, so it is not naturally suited to **categorical data**.
- The algorithm minimizes the Sum of Squared Errors (SSE) but can get trapped in a **local minimum** rather than reaching the global minimum SSE.

(b) K-means may not reach the global optimum – 2.75 marks

Question: Use an example to show why k-means algorithm may not find the global optimum that is optimizing the within-cluster variation.

Solution:

This is the classical illustration used in most Data Mining texts: suppose the true (natural) data actually consists of **three well-separated pairs of sub-clusters** laid out in a row, and we want $K = 3$ clusters overall – the globally optimal solution (minimum total SSE) is to place one centroid at each of the three natural pairs.

However, k-means's final result depends entirely on the **random initial centroids**: if, by chance, **two** of the three initial centroids happen to fall inside the *same* natural pair while the *third* natural pair receives no initial centroid nearby, then:

- The pair containing two initial centroids gets **split** into two separate final clusters (over-segmenting one true group).
- The two natural pairs that are left with only one nearby initial centroid get **merged/absorbed** into a single cluster (under-segmenting two true groups).

Because reassignment in subsequent iterations only ever *locally* reduces SSE (points move to whichever current centroid is nearest), the algorithm settles into this configuration and never "discovers" the better 3-pair partition – it converges to a **local minimum of SSE that is higher (worse)** than the true global minimum. Running k-means multiple times with different random initializations and keeping the run with the lowest SSE is the standard practical remedy, since it increases the chance that at least one run starts close to the globally optimal partition.

(c) Clustering as a preprocessing tool – 3 marks

Question: Clustering is recognized as an important data-mining task with broad applications. Give one application example that uses clustering as a preprocessing tool for other data mining tasks.

Solution:

Example – Customer segmentation before churn classification. A telecom or e-commerce company first runs a clustering algorithm (e.g., k-means) on raw customer behaviour data (usage patterns, spend, tenure, etc.) to group customers into a small number of **behavioural segments**. This cluster label is then added as a single, compact new feature (or used to build one classifier per segment) and fed into a downstream **classification model** that predicts customer churn. Clustering acts as a

preprocessing step that summarizes many raw, noisy attributes into one meaningful categorical feature, reducing dimensionality/complexity and often improving the accuracy and interpretability of the subsequent classifier. (Other common examples: clustering pixels for **image segmentation** before object recognition, or clustering to detect and remove **outliers/noise** before training a classifier.)

Q0.8 Support Vector Machine

(a) Maximum margin classification – 2.75 marks

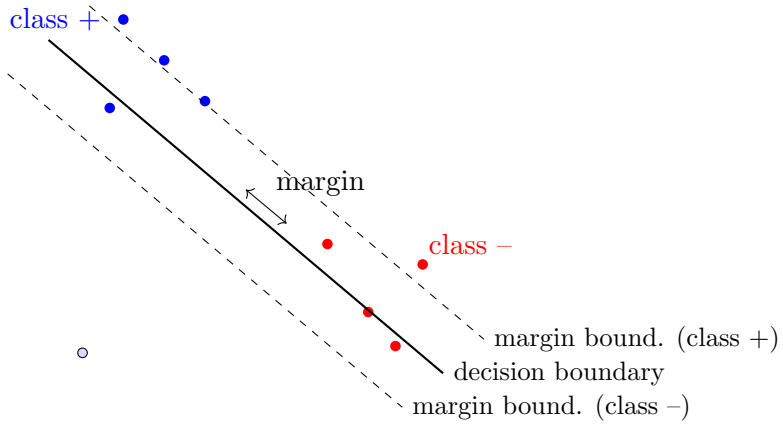
Question: Support Vector Machine always tries to maintain maximum margin for classification. Explain it with proper sketch.

Solution:

Given linearly separable data from two classes, there are, in general, infinitely many hyperplanes that separate them. SVM chooses the one particular hyperplane $w \cdot x + b = 0$ that maximizes the **margin** – the perpendicular distance between the decision boundary and the closest training points from either class (the **support vectors**). The margin width is $\frac{2}{\|w\|}$, so SVM training solves:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1 \ \ \forall i$$

Maximizing the margin (rather than picking any arbitrary separating hyperplane) gives the classifier the best possible **generalization** on unseen data, because it stays as far as possible from both classes and is least sensitive to small perturbations of the data.



(b) Support vectors and high dimensionality – 2 marks

Question: What would be the best generalization for choosing the number of support vectors, if the dimensionality of data is high?

Solution:

The generalization (test) error of an SVM is upper-bounded by the ratio of the number of support vectors to the total number of training examples:

$$E[\text{test error}] \leq \frac{\text{number of support vectors}}{\text{number of training examples}}$$

As dimensionality grows, model complexity (and the risk of overfitting) tends to increase, so the best strategy is to prefer models with as **few support vectors as possible relative to the size of the training set**. A smaller support-vector-to-sample ratio indicates a simpler, more compact decision boundary and, by this bound, better expected generalization – especially important in high dimensions where overfitting is a bigger risk.

(c) SVM classification with given support vectors – 4 marks

Question: Positively labeled points: $\{(4,1),(4,-1),(5,1),(5,-1)\}$. Negatively labeled points: $\{(2,0),(0,2),(0,-2),(-2,0)\}$. Support vectors are $\{(2,0),(4,1),(4,-1)\}$. Apply SVM to classify $(6,-1)$.

Solution:

The decision function has the form $f(x) = w \cdot x + b$, and at the support vectors the margin constraint holds with equality: $y_i(w \cdot x_i + b) = 1$.

By symmetry of the three support vectors about the x_1-axis – $(4,1)$ and $(4,-1)$ are mirror images, and $(2,0)$ lies on the axis – we expect the optimal separating hyperplane to be a **vertical line**, i.e. $w = (w_1, 0)$.

Using the support-vector equations ($y = +1$ for $(4,1)/(4,-1)$, $y = -1$ for $(2,0)$):

$$w_1(4) + 0(1) + b = 1 \;\Rightarrow\; 4w_1 + b = 1$$

$$w_1(4) + 0(-1) + b = 1 \;\Rightarrow\; 4w_1 + b = 1 \quad \text{(same equation – consistent)}$$

$$w_1(2) + 0(0) + b = -1 \;\Rightarrow\; 2w_1 + b = -1$$

Solving the two distinct equations:

$$4w_1 + b = 1, \qquad 2w_1 + b = -1$$

Subtracting: $2w_1 = 2 \;\Rightarrow\; w_1 = 1$, then $b = 1 - 4(1) = -3$.

So $w = (1, 0)$, $b = -3$, and the decision function is:

$$f(x_1, x_2) = x_1 - 3$$

Verification: $f(4,1) = 1$, $f(4,-1) = 1$, $f(2,0) = -1$ – all support vectors sit exactly on their margins, as required.

Classify $(6,-1)$**:**

$$f(6,-1) = 6 - 3 = 3 > 0$$

Since $f(6,-1) = 3 > 0$, $(6,-1)$ is classified as *Positive*.